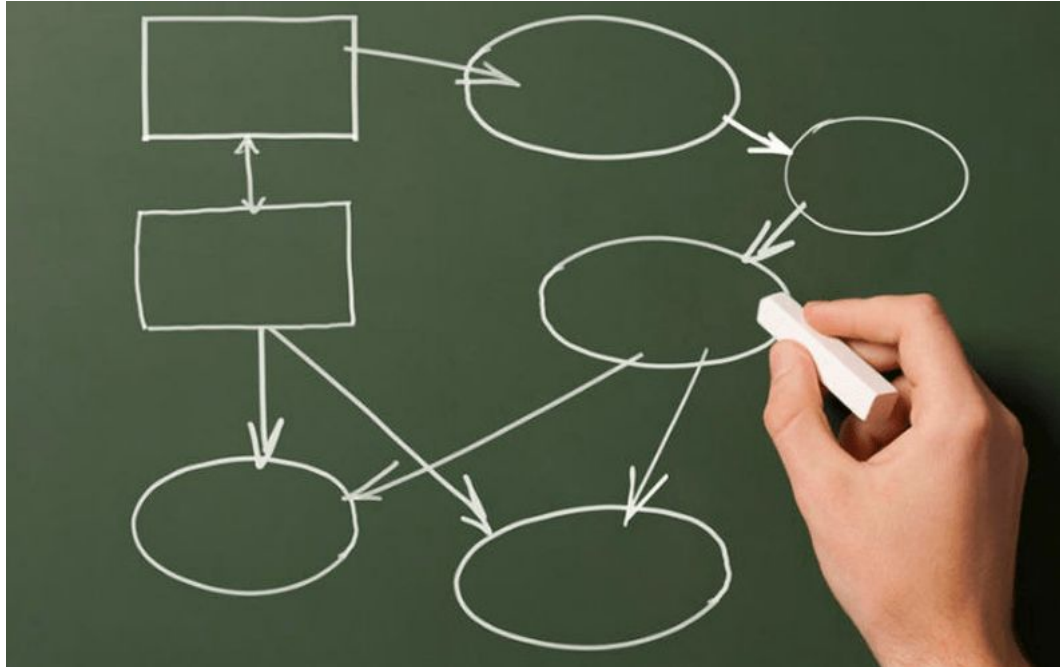
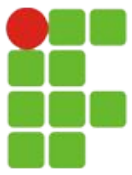
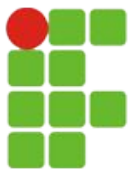




# Algoritmos e programação– ALG

Prof. Jones



## Recordando ....

Trabalhamos na última aula com *Strings*:

O tipo *String* trabalha com qualquer caractere não numérico, ou seja, textos, frases e tipos de dados classificados como não numéricos.

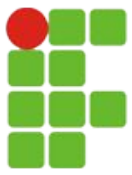
Por exemplo:

O nome de uma pessoa: **Juca Xaves da Silva**

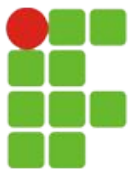
O endereço de uma pessoa: **Av. São João, 631, Barretos/SP**

E-mail de uma pessoa: **jucaxaves@gmail.com**

Podemos fazer manipulações usando estes dados, como por exemplo, mostrar somente o último sobrenome de: **Juca Xaves da Silva**, seria: **Silva** □ na aula vimos como aplicar isso utilizando o comando **substr()**



# Estruturas sequenciais



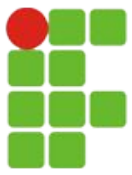
## O que vimos até hoje

### Algoritmos sequenciais

- Problemas sequenciais: as instruções são executadas na ordem em que foram definidas, uma após a outra, sem exceção.

Geralmente incluem três atividades:

- Entrada de dados;
- Processamento realizado sobre os dados de entrada;
- Saída de dados ou apresentação dos resultados.



## O que vimos até hoje

### Exemplo de um algoritmo sequencial

- Faça um programa que calcule a média de 2 notas de um aluno:

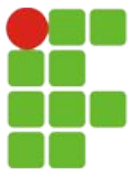
Estudo do enunciado:

- Entradas:  $n1, n2$
- Saída: média
- Operação:  $\text{média} = (n1 + n2) / 2$

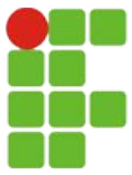
```
#include <iostream>

using namespace std;

int main()
{
    float n1, n2, media;
    cin >> n1 >> n2;
    media = (n1+n2)/2;
    cout << "Media igual: " << media << endl;
    return 0;
}
```



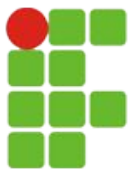
# Estruturas de decisão



# Aula de hoje

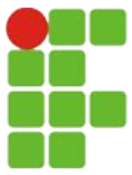
## Estruturas de decisão

Assim como os **seres humanos** precisam tomar decisões para resolver um determinado problema, o **computador** também age seguindo este mesmo critério.



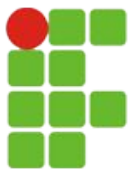
## Por exemplo



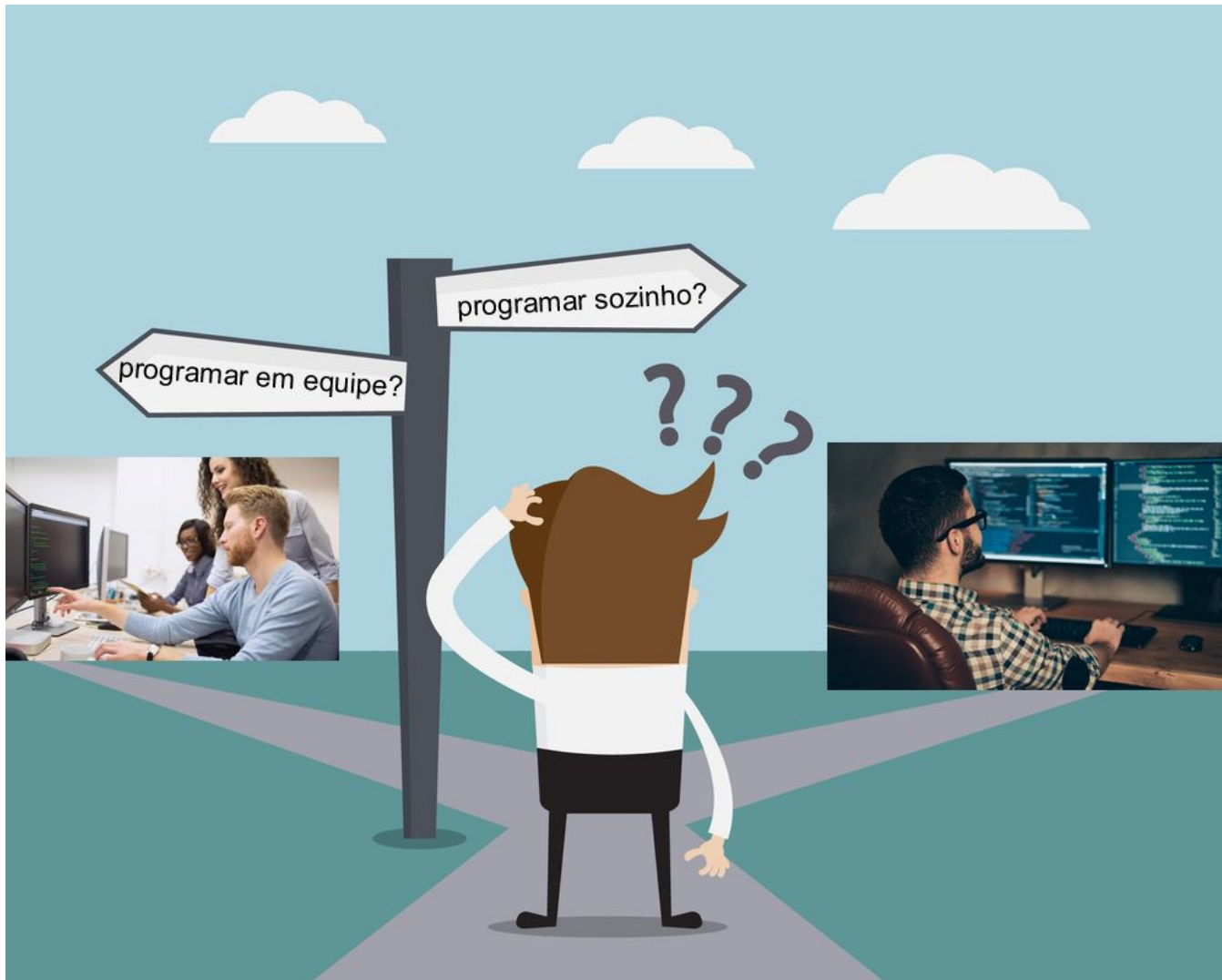


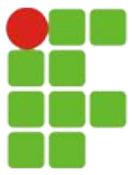
## Por exemplo



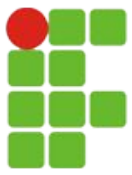


## Por exemplo





# O que é uma estrutura de decisão?

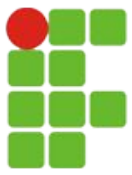


## Estrutura de decisão

As **Estrutura de decisão** permitem ao **programador** imputar um **condição** ao longo do programa. Esta decisão é baseada numa condição que assume ser **verdadeira** ou **falsa**.

Nos problemas condicionais uma ou mais instruções **podem ou não** ser executadas, dependendo da avaliação prévia de **condições**.

Para resolver estes problemas, veremos novos comandos que possibilitam alterar o **fluxo sequencial** de execução de um programa.

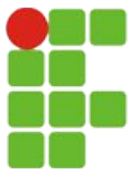


## Estrutura de decisão

As estruturas de decisão dependem diretamente do uso de **operadores relacionais**, por isso, vamos recordar.

### Tabela de operadores relacionais

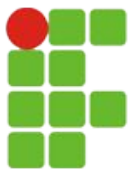
| Operador | Descrição          | Exemplo de utilização | Resultado  |
|----------|--------------------|-----------------------|------------|
| ==       | Igual              | 2==2                  | Verdadeiro |
| !=       | Diferente          | 3!=3                  | Falso      |
| >        | Maior que          | 4>3                   | Verdadeiro |
| <        | Menor que          | 10<5                  | Falso      |
| >=       | Maior ou igual que | 10>=10                | Verdadeiro |
| <=       | Menor ou igual que | 15<=3                 | Falso      |



## Estrutura de decisão

Dependem também do uso de **operadores lógicos**, por isso, vamos recordar.

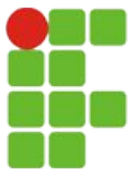
| Descrição | Operador | Exemplo         |
|-----------|----------|-----------------|
| Não       | !        | $!F == V$       |
| OU        |          | $V    F == V$   |
| E         | &&       | $V \&\& F == F$ |



# Operadores lógicos

Considere 0 = FALSO e 1 = VERDADEIRO

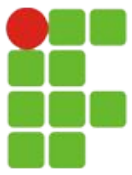
|   |   | E   | OU | não A não B |    |
|---|---|-----|----|-------------|----|
| A | B | AND | OR | !A          | !B |
| 0 | 0 | 0   | 0  | 1           | 1  |
| 0 | 1 | 0   | 1  | 1           | 0  |
| 1 | 0 | 0   | 1  | 0           | 1  |
| 1 | 1 | 1   | 1  | 0           | 0  |



## Forma didática de aprender a tabela verdade

Vamos imaginar a seguinte situação:  
Eu estudo no **IFSP** e quero saber se fui **aprovado** ou **reprovado**.

| Tabela <b>AND ( E )</b>                                                             |                                                                                     |                                                                                       |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Nota                                                                                | Frequência                                                                          | Resultado                                                                             |
|    |    |    |
|   |   |   |
|  |  |  |
|  |  |  |

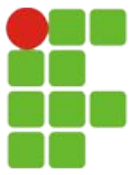


## Forma didática de aprender a tabela verdade

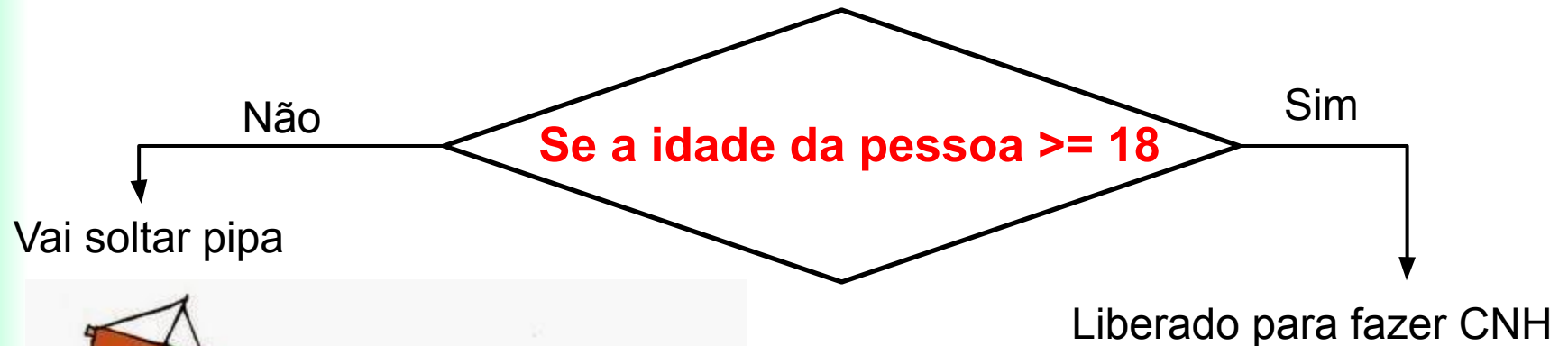
Agora vamos imaginar a seguinte situação:

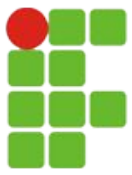
Eu estudo no **ESTADÃO** e quero saber se fui **aprovado** ou **reprovado**.

| Tabela <b>OR (OU)</b>                                                               |                                                                                     |                                                                                       |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Nota                                                                                | Frequência                                                                          | Resultado                                                                             |
|    |    |    |
|   |   |   |
|  |  |  |
|  |  |  |



## Exemplos reais usando os operadores em decisão

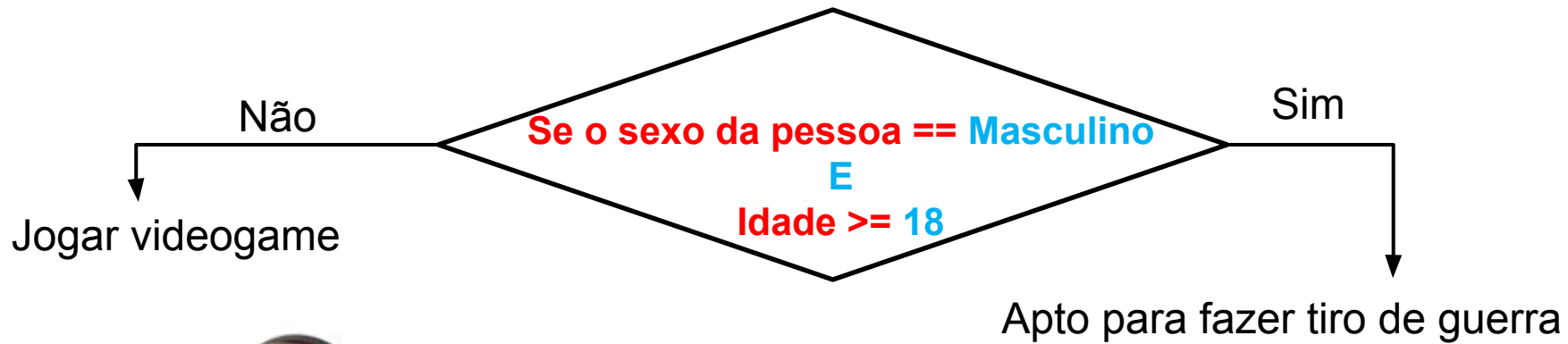


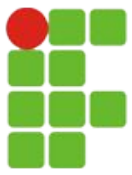


## Exemplos reais usando os operadores em decisão

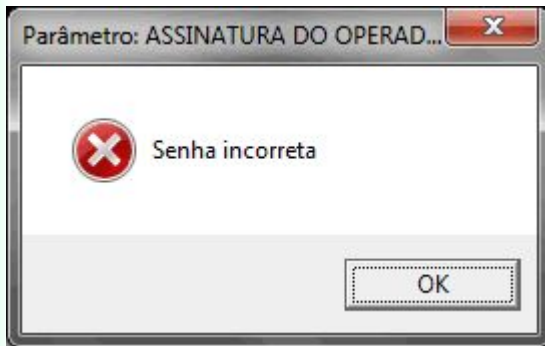
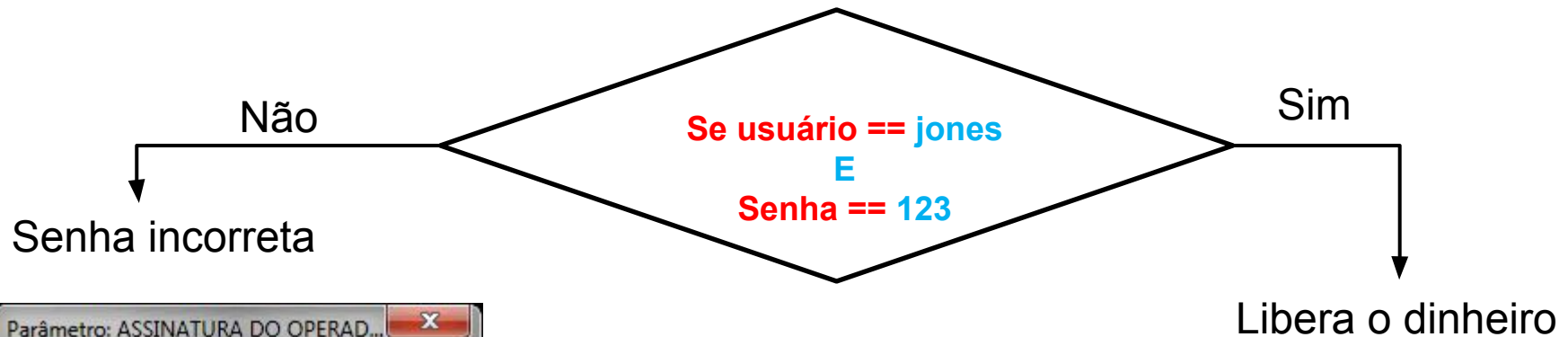


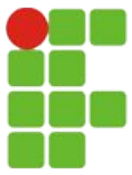
## Exemplos reais usando os operadores em decisão



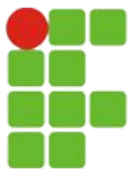


## Exemplos reais usando os operadores em decisão



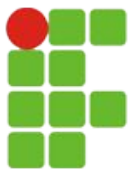


# Comandos de decisão



## Comandos de decisão

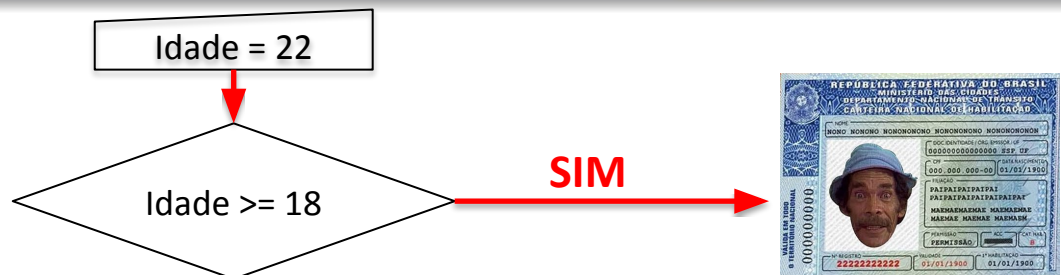
- ✓ **Desvio simples (SE)**
- ✓ **Desvio composto (SE / SENÃO)**
- ✓ **Desvios encadeados (SE / SENÃO SE)**
- ✓ **Desvio com condição múltipla (CASO)**



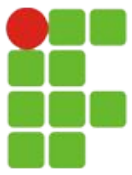
## Introdução ao comando “SE”

A “pergunta” é feita através do comando **SE**.

O comando verifica o conteúdo de uma **variável** e caso o teste lógico retorne **VERDADEIRO** (ou SIM), o comando é executado.



Caso o teste lógico retorne **FALSO** (ou NÃO), o comando não é executado e o programa continua após o término da instrução.



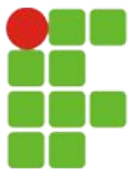
## Comando *if* na linguagem C++

Comando de decisão ou seleção **SIMPLES ( if )**: seleção de instruções que devem ser executadas quando uma condição for **verdadeira**

### Sintaxe:

```
if (<condição>)  
{  
    [bloco de instruções]  
}
```

Serão executados  
somente  
se a condição  
for **VERDADEIRA**



## Desempenho na codificação

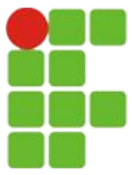
A maioria dos programadores ganham por **hora trabalhada/produção**. Por isso, quanto mais ele produzir, maior será a sua renda. A simplificação de estruturas é importante. O comando de seleção **SIMPLES ( if )** pode ser utilizado da seguinte maneira:

### Até 1 linha de instrução

```
if (<condição>)  
    [instrução única]
```

### A partir de 2 ou mais linhas de instrução

```
if (<condição>)  
{  
    [instrução1]  
    [instrução2]  
    .  
    .  
}
```



## Exemplo 1

Crie um programa que realiza o cálculo da média de **2 notas** de um aluno. Se a média estiver **acima de 6.0**, informe que o aluno foi **aprovado**.

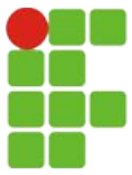
Estudo do enunciado:

- ✓ Entradas: n1, n2
- ✓ Saídas: média, **mensagem aprovado**
- ✓ Operação:  $\text{média} = (n1 + n2) / 2$

```
#include <iostream>

using namespace std;

int main() {
    float n1, n2, media;
    cin >> n1 >> n2;
    media = (n1+n2)/2;
    if(media >= 6.0)
    {
        cout << "Aprovado" << endl;
    }
    return 0;
}
```



## Exemplo 1

```
#include <iostream>

using namespace std;

int main() {
    float n1, n2, media;
    cin >> n1 >> n2;
    media = (n1+n2)/2;
    if (media >= 6.0)
    {
        cout << "Aprovado" << endl;
    }
    return 0;
}
```

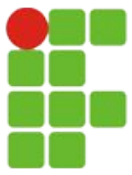
Exemplos  
de Saídas:

```
8.5
6.0
Media = 7.25
Aprovado
```

```
5.5
6.0
Media = 5.75
```

*Condição, expressão lógica*

*Instrução que será executada se a  
condição for verdadeira*



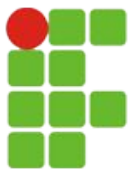
## Exemplo 2

Verificar a situação de um jovem que deseja saber se poderá ou não tirar a carteira de habilitação.

```
#include <iostream>

using namespace std;

int main() {
    int idade;
    cin >> idade;
    if(idade >= 18)
    {
        cout << "Pode tirar CNH" << endl;
    }
    return 0;
}
```



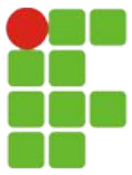
## Exemplo 3

Faça um programa que solicita a idade de uma pessoa e verifica se ela pode **votar** ( $\text{idade} \geq 16$ ).

```
#include <iostream>

using namespace std;

int main() {
    int idade;
    cin >> idade;
    if(idade >= 16)
    {
        cout << "Pode votar!" << endl;
    }
    return 0;
}
```



## Exemplo 4

Faça um programa para mostrar na tela “Acesso permitido”, caso o usuário e senha informados pelo sistema esteja correto.

```
#include <iostream>

using namespace std;

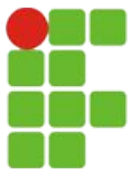
int main()
{
    string usuario, senha;
    getline(cin, usuario);
    getline(cin, senha);

    if(usuario.compare("jones")==0 && senha.compare("123")==0) {
        cout << "Acesso permitido" << endl;
    }

    return 0;
}
```

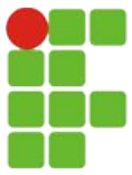
Operador lógico E





## Comandos de decisão

- ✓ **Desvio simples (SE)**
- ✓ **Desvio composto (SE / SENÃO)**
- ✓ **Ternário**
- ✓ **Desvios encadeados (SE / SENÃO SE)**
- ✓ **Desvio com condição múltipla (CASO)**



## Desvio composto

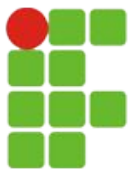
O desvio composto possui a cláusula **SENÃO**.

Desta maneira , podemos tratar o CASO **VERDADEIRO** e o caso **FALSO** do problema.

Exemplos:

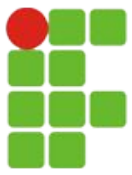
Crie um programa que mostre se o aluno foi **APROVADO** ou **REPROVADO**.

Crie um programa para verificar se um número informado pelo usuário é **PAR** ou **ÍMPAR**.



# Desvio Composto





## Comando *if else* na linguagem C++

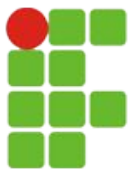
Comando de decisão ou seleção **COMPOSTO ( if )**: seleção de instruções que devem ser executadas quando uma condição for **verdadeira** e caso contrário **falsa**.

### Sintaxe:

```
if (<condição>)  
{  
    [bloco de instruções 1]  
}  
else  
{  
    [bloco de instruções 2]  
}
```

Serão executados  
quando a condição  
for **VERDADEIRA**

Serão executados  
quando a condição  
for **FALSA**



## Desempenho na codificação

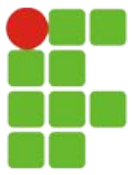
A maioria dos programadores ganham por **hora trabalhada/produção**. Por isso, quanto mais ele produzir, maior será a sua renda. A simplificação de estruturas é importante. O comando de seleção **SIMPLES ( if )** pode ser utilizado da seguinte maneira:

### Até 1 linha de instrução

```
if (<condição>)  
    [instrução única]
```

### A partir de 2 ou mais linhas de instrução

```
if (<condição>)  
{  
    [instrução1]  
    [instrução2]  
    .  
    .  
}
```



## Exemplo 1

Crie um programa que realiza o cálculo da média de **2 notas** de um aluno. Se a média estiver **acima de 6.0**, informe que o aluno foi **aprovado**, senão informe que o aluno foi **reprovado**.

Estudo do enunciado:

- Entradas: n1, n2

- Saídas: média,

Aprovado

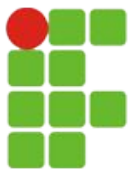
Reprovado

- Operação:  $\text{média} = (n1 + n2) / 2$

```
#include <iostream>

using namespace std;

int main() {
    float N1, N2, Media;
    cin >> N1 >> N2;
    media = (N1+N2)/2;
    if (media >= 6)
    {
        cout << "Aprovado" << endl;
    }
    else
    {
        cout << "Reprovado" << endl;
    }
    return 0;
}
```



## Exemplo 1

- Composto (possui a cláusula senão):

Exemplos  
de Saídas:



```
8.3
6.5
Media = 7.40
Aprovado
```

```
5.2
4.8
Media = 5.00
Reprovado
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    float N1, N2, Media;
```

```
    cin >> N1 >> N2;
```

```
    media = (N1+N2)/2;
```

```
    if (media >= 6)
```

```
    {
```

```
        cout << "Aprovado" << endl;
```

```
    }
```

```
    else
```

```
    {
```

```
        cout << "Reprovado" << endl;
```

```
    }
```

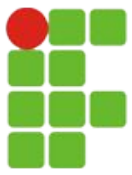
```
    return 0;
```

```
}
```

Condição, expressão lógica

Instrução que será executada se a  
condição for VERDADEIRA

Instrução que será executada se a  
condição for FALSA



## Exemplo 2

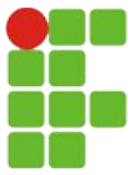
Verificar a situação de um jovem que deseja saber se poderá ou não tirar a carteira de habilitação. Mostre a mensagem **SIM** caso verdadeiro e **NAO** caso não seja possível tirar a CNH.

```
#include <iostream>

using namespace std;

int main() {
    int Idade;
    cin >> Idade;

    if (idade >= 18)
    {
        cout << "SIM" << endl;
    }
    else
    {
        cout << "NAO" << endl;
    }
    return 0;
}
```



## Exemplo 3

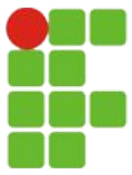
Faça um programa que solicita a idade de uma pessoa e verifica se ela pode votar, caso possa votar exiba a mensagem **PODE VOTAR**, senão exiba a mensagem **NAO PODE VOTAR**.

```
#include <iostream>

using namespace std;

int main() {
    int Idade;
    cin >> Idade;

    if (idade >= 16)
    {
        cout << "PODE VOTAR" << endl;
    }
    else
    {
        cout << "NAO PODE VOTAR" << endl;
    }
    return 0;
}
```



## Exemplo 4

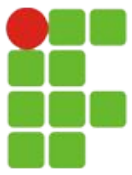
Faça um programa para mostrar na tela **“ACESSO PERMITIDO”**, caso o usuário e senha informados pelo sistema estejam corretos, ou **“ACESSO NEGADO”** caso contrário.

```
#include <iostream>

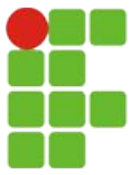
using namespace std;

int main() {
    string usuario, senha;
    cin >> usuario >> senha;

    if (usuario.compare("jones")== 0 && senha.compare("123")==0)
    {
        cout << "ACESSO PERMITIDO" << endl;
    }
    else
    {
        cout << "ACESSO NEGADO" << endl;
    }
    return 0;
}
```



# Operador ternário



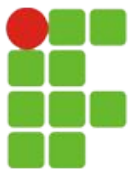
## Operador ternário

Um operador **ternário** consiste numa operação com 3 operandos. Sendo assim, ele trabalha por meio de **três elementos** informados. Nesse caso, trata-se do operador **?** (interrogação).

O operador ternário é equivalente a uma estrutura **IF ELSE**, tratando unicamente 2 condições. **O DESEMPENHO é o mesmo do IF ELSE.**

Forma sintática:

**(condição) ? [ação p/ condição verdadeira] : [ação p/ condição falsa]**



## Exemplo de utilização do operador ternário

Faça um programa para compra de ingressos na Festa do peão de Barretos. O programa deve pedir a idade do cliente e verificar SE for maior de idade (tem 18 anos ou mais), exiba a palavra **OK**. Se tiver menos, seu programa deve exibir a mensagem **NAO**.

```
#include <iostream>
using namespace std;

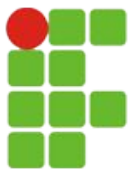
int main()
{
    int idade;
    cin >> idade;

    idade >= 18 ? (cout << "OK") : (cout << "NAO");

    return 0;
}
```

Diagram illustrating the ternary operator usage:

- The condition `idade >= 18` is labeled as **condição**.
- If the condition is **Verdadeiro** (True), the program executes `(cout << "OK")`.
- If the condition is **Falso** (False), the program executes `(cout << "NAO")`.



## Exemplo de utilização do operador ternário

Pode atribuir o resultado da operação ternária a uma variável, por exemplo:

```
#include <iostream>
using namespace std;

int main()
{
    int idade;
    string resposta;
    cin >> idade;

    resposta = idade >= 18 ? "OK": "NAO";
    cout << resposta << endl;

    return 0;
}
```

# Exercícios

Para cada exercício, fazer o **programa em C++** correspondente:

1. Faça um programa que o usuário digite um valor e caso o valor seja positivo, mostre a mensagem “MUITO BEM”.
2. Faça um programa que o usuário digite dois valores e se o segundo valor for maior que o primeiro, mostre a mensagem “ACERTOU”.
3. Faça um algoritmo para ler dois números inteiros e informar se estes são iguais ou diferentes. Mostre a mensagem “SIM”, caso sejam diferentes.
4. Faça um algoritmo para ler um número e mostrar SE E SOMENTE SE este número é PAR, escreva “PAR”, caso verdadeiro.
5. Faça um programa para validar a uma senha informada. Avalie se os caracteres estão em letras maiúsculas.

**7.** Uma empresa decide dar um aumento de 30% aos funcionários com salários inferiores a R\$ 500,00. Faça um programa que receba o salário do funcionário e mostre o valor do salário reajustado ou uma mensagem, caso o funcionário não tenha direito ao aumento.

**8.** Faça um programa para calcular e mostrar o salário reajustado de um funcionário. Sabe-se que o percentual de aumento é o mesmo da tabela a seguir.

| SALÁRIO             | PERCENTUAL DE AUMENTO |
|---------------------|-----------------------|
| Até R\$ 300,00      | 35%                   |
| Acima de R\$ 300,00 | 15%                   |

**15.** Uma agência bancária possui dois tipos de investimentos, conforme o quadro a seguir. Faça um programa que receba o tipo de investimento e o valor do investimento e que calcule e mostre o valor corrigido de acordo com o tipo de investimento.

| TIPO | DESCRIÇÃO            | RENDIMENTO MENSAL |
|------|----------------------|-------------------|
| 1    | Poupança             | 3%                |
| 2    | Fundos de renda fixa | 4%                |

**17.** Faça um programa que verifique a validade de uma senha fornecida pelo usuário. A senha é 4531. O programa deve mostrar uma mensagem de permissão de acesso ou não.

**18.** Faça um programa que receba a idade de uma pessoa e mostre a mensagem de maioridade ou não.

**19.** Faça um programa que receba a altura e o sexo de uma pessoa e que calcule e mostre o seu peso ideal, utilizando as seguintes fórmulas:

- ♦ para homens:  $(72.7 * h) - 58$ ;
- ♦ para mulheres:  $(62.1 * h) - 44.7$